MODULE 5.2 PROGRAMMING ASSIGNMENT

GitHub Repository Link: https://github.com/devcnx/csd440

BRITTANEYCUSTOMERS.PHP

```php
<?php
/**
 * Module 5.2 Programming Assignment
 * CSD440 Server-Side Scripting
 *
 * This program builds a multidimensional associative array of customer
 * records (first name, last name, age, phone number) and demonstrates
 * several PHP array methods for searching and retrieving records by
 * different data fields.
 *
 * Array techniques demonstrated:
 *   - array_filter()  : retrieve all customers matching a predicate
 *   - array_column()  : extract a single field across every record
 *   - array_search()  : locate the index of a record by field value
 *   - in_array()      : check for the existence of a value
 *   - usort()         : sort the customer list by a chosen field
 *   - count()         : report the size of the dataset and filtered subsets
 *
 * Requires PHP 8.0 or newer. The script uses language features that are
 * unavailable on older interpreters, specifically:
 *   - str_starts_with() (PHP 8.0+) in findCustomersByAreaCode()
 *   - Typed properties, union return types, and arrow functions (PHP 7.4+)
 *   - The spaceship operator <=> (PHP 7.0+)
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-04-19
 * @php     >= 8.0
 */

/**
 * Returns the customer record whose last name matches the supplied value.
 *
 * Uses array_column() to build a lookup array of last names, then
 * array_search() to find the matching index. Returns null when no
 * customer is found so the caller can handle the miss gracefully.
 *
 * @param array  $customers The full customer dataset
 * @param string $lastName  The last name to search for (case-insensitive)
 * @return array|null       The matched customer record, or null if absent
 */
function findCustomerByLastName(array $customers, string $lastName): ?array
{
    $lastNames = array_map('strtolower', array_column($customers, 'last_name'));
    $index = array_search(strtolower($lastName), $lastNames, true);
    return $index === false ? null : $customers[$index];
}
```

```php
/**
 * Returns every customer whose age falls within the given inclusive range.
 *
 * Uses array_filter() with a closure so the age bounds can be passed in
 * at call time. array_values() reindexes the filtered result so it can
 * be iterated with a standard foreach without gaps in the keys.
 *
 * @param array $customers The full customer dataset
 * @param int   $minAge    The minimum age (inclusive)
 * @param int   $maxAge    The maximum age (inclusive)
 * @return array           Customers whose age is between $minAge and $maxAge
 */
function findCustomersInAgeRange(array $customers, int $minAge, int $maxAge): array
{
    $filtered = array_filter(
        $customers,
        fn(array $c) => $c['age'] >= $minAge && $c['age'] <= $maxAge
    );
    return array_values($filtered);
}

/**
 * Returns every customer whose phone number uses the given area code.
 *
 * Phone numbers are stored as "(XXX) XXX-XXXX" so the area code is the
 * substring between the parentheses. Using str_starts_with() keeps the
 * match anchored to the beginning of the string.
 *
 * @param array  $customers The full customer dataset
 * @param string $areaCode  The 3-digit area code to match (e.g., "402")
 * @return array            Customers whose phone number starts with the area code
 */
function findCustomersByAreaCode(array $customers, string $areaCode): array
{
    $prefix = "($areaCode)";
    $filtered = array_filter(
        $customers,
        fn(array $c) => str_starts_with($c['phone'], $prefix)
    );
    return array_values($filtered);
}

/**
 * Returns a copy of the customer list sorted by the given field.
 *
 * usort() mutates the array it operates on, so the input is copied first
 * to preserve the caller's original ordering. The comparator uses the
 * spaceship operator (<=>) which handles both string and numeric fields.
 *
 * @param array  $customers The customer dataset to sort
 * @param string $field     The record key to sort by (e.g., 'age', 'last_name')
 * @return array            A new array sorted ascending by the chosen field
```

```
 */
function sortCustomersByField(array $customers, string $field): array
{
   $sorted = $customers;
   usort($sorted, fn(array $a, array $b) => $a[$field] <=> $b[$field]);
   return $sorted;
}

/**
 * Renders a customer table from an array of records.
 *
 * Defined inline because it is only used by this page. Escapes every
 * value with htmlspecialchars() so that any unexpected characters in
 * a record field cannot break the HTML or introduce XSS.
 *
 * @param array  $rows    Customer records to render
 * @param string $caption Optional table caption
 * @return void
 */
function renderCustomerTable(array $rows, string $caption = ''): void
{
   if (empty($rows)) {
      echo "<p class=\"empty\">No customers matched this query.</p>";
      return;
   }
   echo "<table>";
   if ($caption !== '') {
      echo "<caption>" . htmlspecialchars($caption) . "</caption>";
   }
   echo "<thead><tr>
         <th>First Name</th>
         <th>Last Name</th>
         <th>Age</th>
         <th>Phone</th>
        </tr></thead><tbody>";
   foreach ($rows as $c) {
      echo "<tr>"
         . "<td>" . htmlspecialchars($c['first_name']) . "</td>"
         . "<td>" . htmlspecialchars($c['last_name']) . "</td>"
         . "<td>" . htmlspecialchars((string) $c['age']) . "</td>"
         . "<td>" . htmlspecialchars($c['phone']) . "</td>"
         . "</tr>";
   }
   echo "</tbody></table>";
}

// Customer dataset — 10 records with first name, last name, age, and phone
$customers = [
   ['first_name' => 'Amelia', 'last_name' => 'Reyes', 'age' => 27, 'phone' => '(402) 555-0183'],
   ['first_name' => 'Marcus', 'last_name' => 'Okafor', 'age' => 41, 'phone' => '(312) 555-0294'],
   ['first_name' => 'Priya', 'last_name' => 'Natarajan', 'age' => 34, 'phone' => '(415) 555-0147'],
   ['first_name' => 'Desmond', 'last_name' => 'Whitaker', 'age' => 52, 'phone' => '(402) 555-0236'],
   ['first_name' => 'Hana', 'last_name' => 'Takeda', 'age' => 23, 'phone' => '(206) 555-0411'],
```

```
    ['first_name' => 'Elias', 'last_name' => 'Brennan', 'age' => 38, 'phone' => '(617) 555-0358'],
    ['first_name' => 'Noemi', 'last_name' => 'Delacroix', 'age' => 45, 'phone' => '(504) 555-0162'],
    ['first_name' => 'Tobias', 'last_name' => 'Kensington', 'age' => 29, 'phone' => '(312) 555-0477'],
    ['first_name' => 'Yuki', 'last_name' => 'Morgan', 'age' => 31, 'phone' => '(402) 555-0519'],
    ['first_name' => 'Imani', 'last_name' => 'Fairchild', 'age' => 36, 'phone' => '(415) 555-0628'],
];

// Perform the lookups and store results for display in the page body
$totalCustomers = count($customers);
$allLastNames = array_column($customers, 'last_name');
$searchLastName = 'Takeda';
$byLastName = findCustomerByLastName($customers, $searchLastName);
$hasWhitaker = in_array('Whitaker', $allLastNames, true);
$ageRangeLow = 30;
$ageRangeHigh = 40;
$inAgeRange = findCustomersInAgeRange($customers, $ageRangeLow, $ageRangeHigh);
$areaCode = '402';
$inAreaCode = findCustomersByAreaCode($customers, $areaCode);
$sortedByAge = sortCustomersByField($customers, 'age');
$sortedByLastName = sortCustomersByField($customers, 'last_name');

// Page metadata
$studentName = 'Brittaney Perry-Morgan';
$assignmentTitle = 'Module 5.2 Programming Assignment';
$courseName = 'CSD440 Server-Side Scripting';
$date = date('F j, Y');
?>
<!DOCTYPE html>
<html lang="en">

    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Customer Records — <?php echo $assignmentTitle; ?></title>
        <style>
            body {
                font-family: Arial, sans-serif;
                max-width: 1000px;
                margin: 40px auto;
                padding: 20px;
                background-color: #f5f5f5;
                color: #222;
            }

            h1 {
                text-align: center;
                color: #2c3e50;
            }

            h2 {
                color: #2c3e50;
                border-bottom: 2px solid #4a90d9;
                padding-bottom: 6px;
```

```
    margin-top: 36px;
}

.header-info {
    text-align: center;
    margin-bottom: 30px;
    color: #555;
}

.query-note {
    background: #eef4fb;
    border-left: 4px solid #4a90d9;
    padding: 10px 14px;
    margin: 12px 0;
    font-size: 0.95em;
}

table {
    width: 100%;
    border-collapse: collapse;
    background-color: #fff;
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.08);
    margin-top: 8px;
}

caption {
    caption-side: top;
    text-align: left;
    padding: 8px 0;
    font-weight: bold;
    color: #2c3e50;
}

th {
    background-color: #2c3e50;
    color: #fff;
    padding: 10px 12px;
    text-align: left;
}

td {
    padding: 10px 12px;
    border-bottom: 1px solid #ddd;
}

tr:hover {
    background-color: #f9f9f9;
}

.empty {
    color: #888;
    font-style: italic;
}
```

```
        .flag-true {
            color: #27ae60;
            font-weight: bold;
        }

        .flag-false {
            color: #c0392b;
            font-weight: bold;
        }
    </style>
</head>

<body>
    <h1>Customer Records</h1>
    <div class="header-info">
        <p><strong><?php echo $studentName; ?></strong></p>
        <p><?php echo $assignmentTitle; ?> — <?php echo $courseName; ?></p>
        <p><?php echo $date; ?></p>
    </div>

    <h2>Full Customer List (<?php echo $totalCustomers; ?> records)</h2>
    <div class="query-note">
        Dataset built as a multidimensional associative array. Each record holds
        <code>first_name</code>, <code>last_name</code>, <code>age</code>, and <code>phone</code>.
    </div>
    <?php renderCustomerTable($customers); ?>

    <h2>Lookup by Last Name — <em>"<?php echo htmlspecialchars($searchLastName); ?>"</em></h2>
    <div class="query-note">
        Uses <code>array_column()</code> to flatten the last-name field across every record,
        then <code>array_search()</code> to locate the matching index.
    </div>
    <?php
    if ($byLastName !== null) {
        renderCustomerTable([$byLastName]);
    } else {
        echo "<p class=\"empty\">No customer found with last name \"" . htmlspecialchars($searchLastName) .
"\".</p>";
    }
    ?>

    <h2>Existence Check — Is there a customer named "Whitaker"?</h2>
    <div class="query-note">
        Uses <code>in_array()</code> against the extracted last-name list. Returns a simple
        boolean rather than a record.
    </div>
    <p>
        Result:
        <span class="<?php echo $hasWhitaker ? 'flag-true' : 'flag-false'; ?>">
            <?php echo $hasWhitaker ? 'YES — a customer with that last name exists.' : 'NO — no match found.'; ?>
        </span>
    </p>
```

```
    <h2>Customers Aged <?php echo $ageRangeLow; ?>–<?php echo $ageRangeHigh; ?></h2>
    <div class="query-note">
      Uses <code>array_filter()</code> with an arrow-function predicate to return every
      record whose <code>age</code> field falls inside the inclusive range.
      Matches: <?php echo count($inAgeRange); ?>.
    </div>
    <?php renderCustomerTable($inAgeRange); ?>

    <h2>Customers in Area Code (<?php echo htmlspecialchars($areaCode); ?>)</h2>
    <div class="query-note">
      Uses <code>array_filter()</code> combined with <code>str_starts_with()</code> to match
      the phone prefix. Matches: <?php echo count($inAreaCode); ?>.
    </div>
    <?php renderCustomerTable($inAreaCode); ?>

    <h2>Sorted by Age (Ascending)</h2>
    <div class="query-note">
      Uses <code>usort()</code> with a spaceship-operator comparator. The original
      <code>$customers</code> array is left untouched by copying before sorting.
    </div>
    <?php renderCustomerTable($sortedByAge); ?>

    <h2>Sorted by Last Name (A–Z)</h2>
    <div class="query-note">
      Same <code>usort()</code> helper, called with a different field name — demonstrates
      that the sort comparator works for strings as well as numbers.
    </div>
    <?php renderCustomerTable($sortedByLastName); ?>
  </body>

</html>
```

DELIVERABLES

Customer Records

Brittaney Perry-Morgan

Module 5.2 Programming Assignment — CSD440 Server-Side Scripting

April 19, 2026

Full Customer List (10 records)

Dataset built as a multidimensional associative array. Each record holds `first_name`, `last_name`, `age`, and `phone`.

First Name	Last Name	Age	Phone
Amelia	Reyes	27	(402) 555-0183
Marcus	Okafor	41	(312) 555-0294
Priya	Natarajan	34	(415) 555-0147
Desmond	Whitaker	52	(402) 555-0236
Hana	Takeda	23	(206) 555-0411
Elias	Brennan	38	(617) 555-0358
Noemi	Delacroix	45	(504) 555-0162
Tobias	Kensington	29	(312) 555-0477
Yuki	Morgan	31	(402) 555-0519
Imani	Fairchild	36	(415) 555-0628

Lookup by Last Name — *"Takeda"*

Uses `array_column()` to flatten the last-name field across every record, then `array_search()` to locate the matching index.

First Name	Last Name	Age	Phone
Hana	Takeda	23	(206) 555-0411

Existence Check — Is there a customer named "Whitaker"?

Uses `in_array()` against the extracted last-name list. Returns a simple boolean rather than a record.

Result: **YES — a customer with that last name exists.**

Customers Aged 30–40

Uses `array_filter()` with an arrow-function predicate to return every record whose `age` field falls inside the inclusive range. Matches: 4.

First Name	Last Name	Age	Phone
Priya	Natarajan	34	(415) 555-0147
Elias	Brennan	38	(617) 555-0358
Yuki	Morgan	31	(402) 555-0519
Imani	Fairchild	36	(415) 555-0628

Customers in Area Code (402)

Uses `array_filter()` combined with `str_starts_with()` to match the phone prefix. Matches: 3.

First Name	Last Name	Age	Phone
Amelia	Reyes	27	(402) 555-0183
Desmond	Whitaker	52	(402) 555-0236
Yuki	Morgan	31	(402) 555-0519

Sorted by Age (Ascending)

Uses `usort()` with a spaceship-operator comparator. The original `$customers` array is left untouched by copying before sorting.

First Name	Last Name	Age	Phone
Hana	Takeda	23	(206) 555-0411
Amelia	Reyes	27	(402) 555-0183
Tobias	Kensington	29	(312) 555-0477
Yuki	Morgan	31	(402) 555-0519
Priya	Natarajan	34	(415) 555-0147
Imani	Fairchild	36	(415) 555-0628
Elias	Brennan	38	(617) 555-0358
Marcus	Okafor	41	(312) 555-0294
Noemi	Delacroix	45	(504) 555-0162
Desmond	Whitaker	52	(402) 555-0236

Sorted by Last Name (A–Z)

Same `usort()` helper, called with a different field name — demonstrates that the sort comparator works for strings as well as numbers.

First Name	Last Name	Age	Phone
Elias	Brennan	38	(617) 555-0358
Noemi	Delacroix	45	(504) 555-0162
Imani	Fairchild	36	(415) 555-0628
Tobias	Kensington	29	(312) 555-0477
Yuki	Morgan	31	(402) 555-0519
Priya	Natarajan	34	(415) 555-0147
Marcus	Okafor	41	(312) 555-0294
Amelia	Reyes	27	(402) 555-0183
Hana	Takeda	23	(206) 555-0411
Desmond	Whitaker	52	(402) 555-0236